

ELI MKXI — Test Coverage Suite and Results

Contents

Headline	1
The suite — 13 new test modules (~200 tests)	1
Coverage by subsystem	2
The 5 pre-existing reds (none from this work)	2
On comparisons (e.g. a “closest-in-spirit” project claiming ~89%)	2

A test-coverage suite was built for ELI and run end-to-end. This records the suite, the methodology, the measured results, and the honest exclusions. Reproducible via `scripts/coverage_full.sh`; methodology in [docs/COVERAGE.md](#).

Headline

Metric	Value
Testable coverage (GUI excluded)	47.1% — 30,431 / 64,587 statements
Raw coverage (whole tree, GUI included)	39.5%
Tests passing	7,193 (unit) + 72 (web) + 15 (live)
Pre-existing reds	5 (unrelated — see below)
Skipped / xfailed	44 / 2 (documented)

The GUI (~12,600 statements) is the only exclusion from the “testable” denominator — it cannot run in headless CI (constructing the window blocks on `display/device` init). Everything else, including hardware-adjacent and untested code, stays in the count; excluding merely-untested code would inflate the figure dishonestly.

The suite — 13 new test modules (~200 tests)

Built across three lanes because the fast unit suite mocks the heavy deps (`pydantic/llama_cpp/torch/faiss`), so the real web server, a real model, and the GUI can’t be exercised in-process:

1. **Mocked unit lane** — breadth + core logic (the existing 7,000+ suite plus the new pure-logic modules below).
2. **Web-server lane** (`--noconftest`, real FastAPI) — `api/server.py`.
3. **Live-engine lane** (`--noconftest`, real GGUF model) — the full pipeline.

New modules: web server (auth/RBAC + endpoints, `api/server.py` 0→53%), live engine integration (real GGUF turns), deterministic grounding gate, news synthesis, executor pure helpers (fail-closed shell gate + hygiene scanners), control contracts, user-visible response surface, live introspection, grounded remediation, memory evidence (12→77%), diagnostic-action classifier, persona handoff builder, operator policy (42→88%). Assertions target real contracts, with a bias to the security/privacy-critical paths (auth, the shell allowlist, PII scanners, the anti-confabulation guards, DB isolation).

Coverage by subsystem

Subsystem	Cov	Subsystem	Cov
eli/onboarding	85%	eli/kernel	51%
eli/coding	81%	eli/runtime	51%
eli/cognition	65%	eli/planning	41%
eli/learning	62%	eli/execution	39%
eli/core	54%	eli/tools	29%
eli/memory	54%	eli/perception	26%
eli/world	54%	eli/utils	23%
api (web server)	51%	eli/integrations	21%

The cognitive core (coding, cognition, learning, kernel, memory, core) sits at 51-81%. The lower areas are the side-effecting handler surface (execution) and the device-bound paths (perception/tools), which only move via live-lane integration turns, not unit tests.

The 5 pre-existing reds (none from this work)

1-3. smart_home plugin — the in-progress Home-Assistant removal (voice SMART_HOME now uses ELI's own MQTT server). 4. A blueprint references a since-moved file (eli/execution/handlers/__init__.py). 5. Silent-swallow ratchet — 987 except: pass vs a 950 ceiling (an observability debt the codebase already carried; the ratchet test correctly forbids raising the ceiling — clear them by making the swallows observable).

On comparisons (e.g. a “closest-in-spirit” project claiming ~89%)

The gap is largely *what ELI is*: it carries a large **untestable embodiment surface** — desktop GUI, gaze/webcam, mic/voice, local vision, OS control, smart-home — that can't run in headless CI. A leaner pure-software agent simply lacks that surface, so a higher fraction of its code is unit-testable by construction. ELI's cognitive *core* is already in a comparable band; the raw number is lower because ELI does more, not because the core is weak. The honest path past 47% is more **live-lane integration tests**, not denominator tricks.